



Instituto Tecnológico de Aeronáutica
Divisão de Engenharia Aeronáutica e Aeroespacial

Laboratório 2

Aprendizado Profundo

Autores:

João Vitor Coelho Guimarães Lima

Professor:

Prof. Marcos Ricardo Omena de Albuquerque Maximo

2 de abril de 2026

1 Introdução

Este laboratório tem como objetivo reproduzir o movimento de caminhada de um robô humanoide por meio de uma técnica conhecida como *imitation learning* (aprendizado por imitação). Para tanto, será utilizado o *framework* de aprendizado profundo PyTorch.

2 Metodologia

Para a replicação do comportamento dinâmico do robô humanoide, aplica-se a técnica de *Imitation Learning* (Aprendizagem por Imitação) via aprendizagem supervisionada. O objetivo é aproximar a política de controle original, baseada em teoria de controle, por meio de uma rede neural artificial implementada no *framework* *PyTorch*.

2.1 Arquitetura da Rede Neural

Para mapear o tempo do ciclo de marcha às posições angulares das juntas do robô, optou-se por uma rede do tipo *Multilayer Perceptron* (MLP). Trata-se de uma arquitetura simples, mas suficiente para o problema de regressão em questão, no qual a entrada é um escalar e a saída, um vetor de 20 dimensões.

A camada de entrada possui um único neurônio, que recebe o instante t dentro do ciclo. Em seguida, há duas camadas ocultas densas, com 75 e 50 neurônios respectivamente, ambas utilizando *LeakyReLU* como função de ativação ($\text{negative_slope} = 0,01$). A escolha da *LeakyReLU* em detrimento da *ReLU* tradicional deve-se ao fato de evitar o problema de neurônios inativos durante o treinamento. Por fim, a camada de saída conta com 20 neurônios de ativação linear, correspondentes às 20 juntas do robô.

O treinamento tem como objetivo reduzir a diferença entre as trajetórias do controlador de referência e as previsões da rede. A Tabela 1 sintetiza a topologia adotada, o diagrama 1 mostra de forma visual a arquitetura.

Tabela 1: Especificações da arquitetura da rede neural.

Camada	Tipo	Neurônios	Ativação
Entrada	Linear	1	-
Oculto 1	Linear	75	LeakyReLU (0,01)
Oculto 2	Linear	50	LeakyReLU (0,01)
Saída	Linear	20	Linear

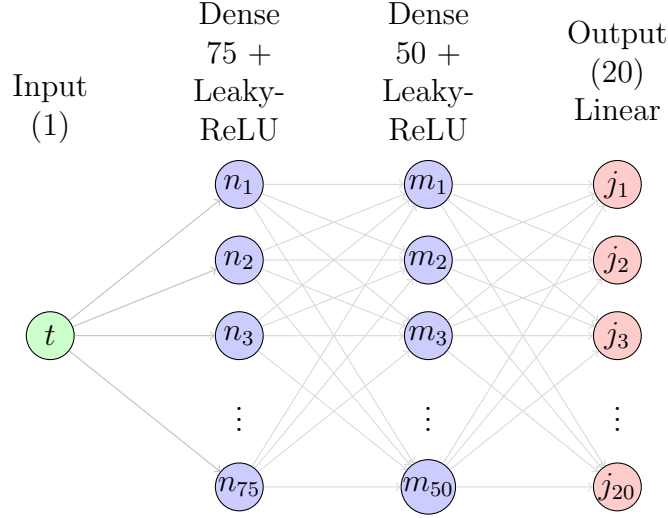


Figura 1: Topologia da rede neural MLP empregada na regressão das trajetórias articulares.

Usando PyTorch, implemente uma rede neural de acordo com a arquitetura apresentada na Tabela 1. Leve em consideração o seguinte para o treinamento:

2.2 Treinamento da Rede

O treinamento foi conduzido sem o uso de técnicas de regularização, empregando o conjunto de dados completo a cada iteração (*full-batch*). Como função de custo, adotou-se o erro quadrático médio (*Mean Squared Error*, MSE), definido por:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (1)$$

em que N é o número de amostras, y_i corresponde ao valor de referência fornecido pelo controlador e $\hat{y}_i$ é o valor predito pela rede. Para o caso multivariado, com 20 saídas correspondentes às juntas do robô, a função custo é estendida como a média do erro quadrático sobre todas as dimensões de saída:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N \cdot D} \sum_{i=1}^N \sum_{d=1}^D (y_{i,d} - \hat{y}_{i,d})^2, \quad (2)$$

sendo $D = 20$ o número de juntas.

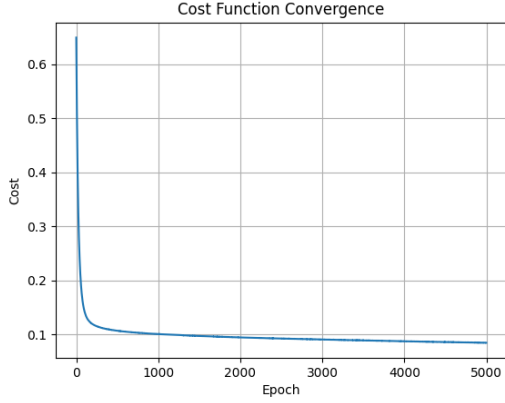
A otimização dos pesos foi realizada com o algoritmo Adam, configurado com taxa de aprendizado fixa de 10^{-4} . O processo de treinamento foi conduzido por 30.000 épocas, valor empiricamente suficiente para a convergência da função custo nas condições do problema.

3 Resultados

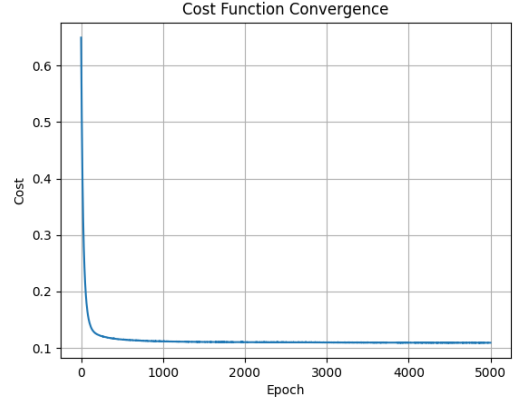
3.1 Influência da Regularização no Treinamento

A aplicação de regularização ao problema da função `sum_gt_zero` ($x_1 + x_2 \geq 0$) levou a um custo final de treinamento superior ao obtido sem regularização. A rede estabilizou

em um patamar mais alto, sem alcançar o mínimo atingido na ausência de regularização. Tal comportamento é coerente com a função do termo regularizador, que penaliza pesos elevados e desloca o ponto de mínimo da função de custo total para fora do mínimo do erro nos dados. Em problemas simples e sem ruído, como o presente caso, não há sobreajuste a ser mitigado, de modo que essa penalização atua apenas como um viés indesejado. A Figura 2 ilustra esse comportamento.



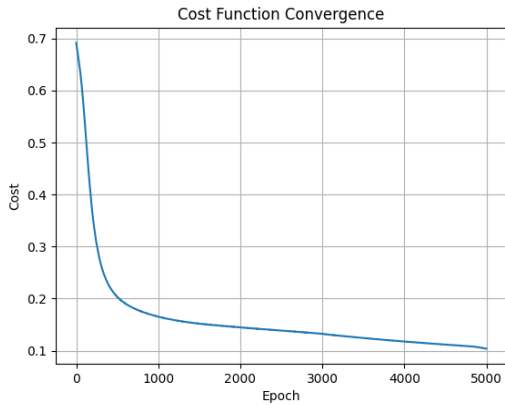
(a) Sem regularização ($\lambda = 0,0$).



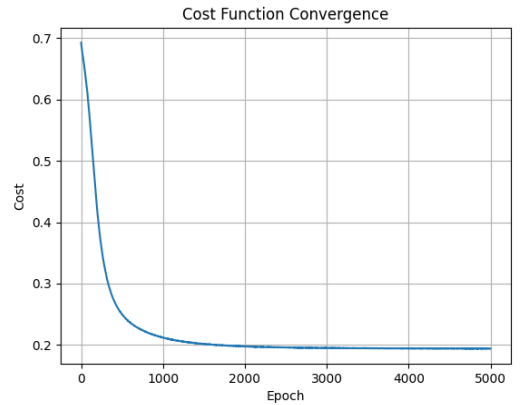
(b) Com regularização ($\lambda = 0,002$).

Figura 2: Curvas de convergência para a função `sum_gt_zero`.

Comportamento análogo foi observado para a função XOR, conforme apresentado na Figura 3: o treinamento com regularização estabiliza em um valor de custo final mais elevado do que o caso sem regularização. Como ambos os problemas são determinísticos e dispõem de dados suficientes, não há ganho em generalização que justifique o uso de regularização nesse contexto.



(a) Sem regularização ($\lambda = 0,0$).



(b) Com regularização ($\lambda = 0,002$).

Figura 3: Curvas de convergência para a função XOR.

A análise restrita à função de custo, no entanto, não captura o efeito completo da regularização. Ao examinar a fronteira de decisão obtida pelas redes treinadas, observa-se que o caso regularizado, embora apresente custo final mais elevado, produz uma classificação com transição mais suave entre as classes. Sem regularização, a rede ajusta-se de forma quase rígida aos dados de treinamento, gerando uma fronteira mais abrupta; com regularização, a superfície aprendida torna-se mais fluida, ainda que admita pequenos erros de

classificação em pontos próximos à fronteira. As Figuras 4 e 5 evidenciam essa diferença qualitativa para a `sum_gt_zero` e o XOR, respectivamente. No caso do XOR, a tendência é particularmente clara: sem regularização, a rede deforma a fronteira de decisão para acomodar todos os pontos do conjunto de treinamento, enquanto a versão regularizada preserva uma geometria mais regular, sem distorcer o ajuste em torno de cada amostra.

Esse comportamento ilustra o trade-off típico introduzido pela regularização: trocar parte do ajuste fino aos dados de treinamento por uma representação mais regular da função aprendida. Em cenários com dados ruidosos ou conjuntos de treino limitados, essa fluidez tende a se traduzir em melhor capacidade de generalização — efeito que, no presente experimento, não chega a se manifestar como vantagem prática, dada a natureza determinística do problema.

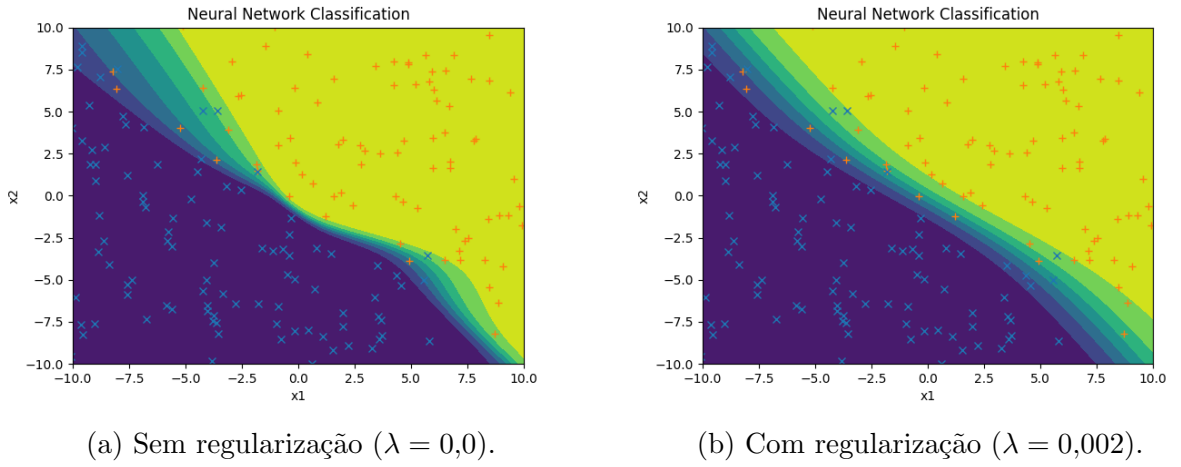


Figura 4: Fronteira de decisão aprendida pela rede para a função `sum_gt_zero`.

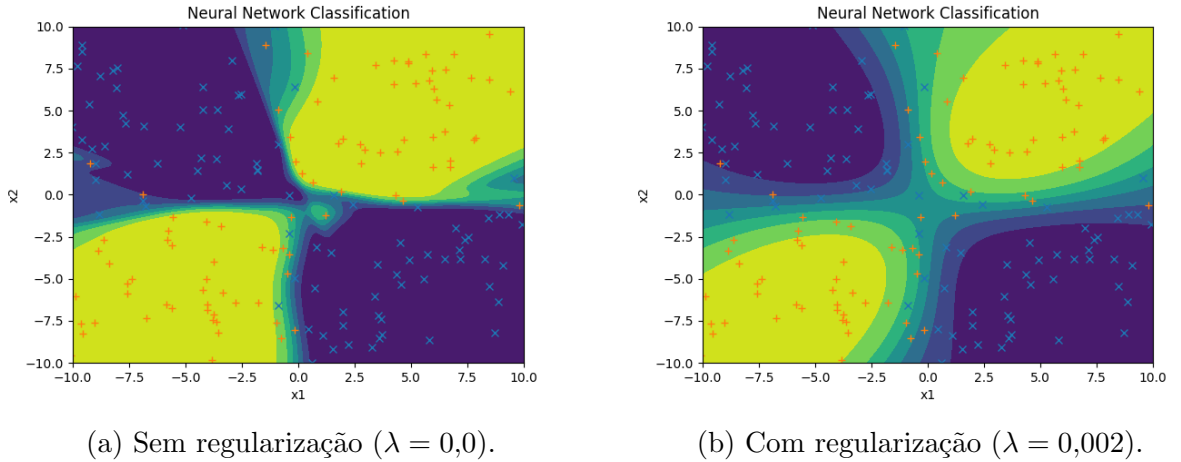


Figura 5: Fronteira de decisão aprendida pela rede para a função XOR.

3.2 Regressão das Trajetórias Articulares

Para o treinamento da rede responsável por reproduzir as trajetórias articulares do robô, optou-se por não utilizar regularização. Como consequência, o ajuste obtido apresenta menor suavidade em comparação com casos regularizados, mas acompanha de perto

o sinal de referência fornecido pelo controlador. Os resultados mostram que a arquitetura adotada é capaz de aproximar adequadamente a função real de movimentação do robô, com convergência satisfatória ao longo do ciclo de marcha. A Figura 6 apresenta a comparação entre as trajetórias de referência e as previsões da rede para cinco juntas representativas do membro inferior direito.

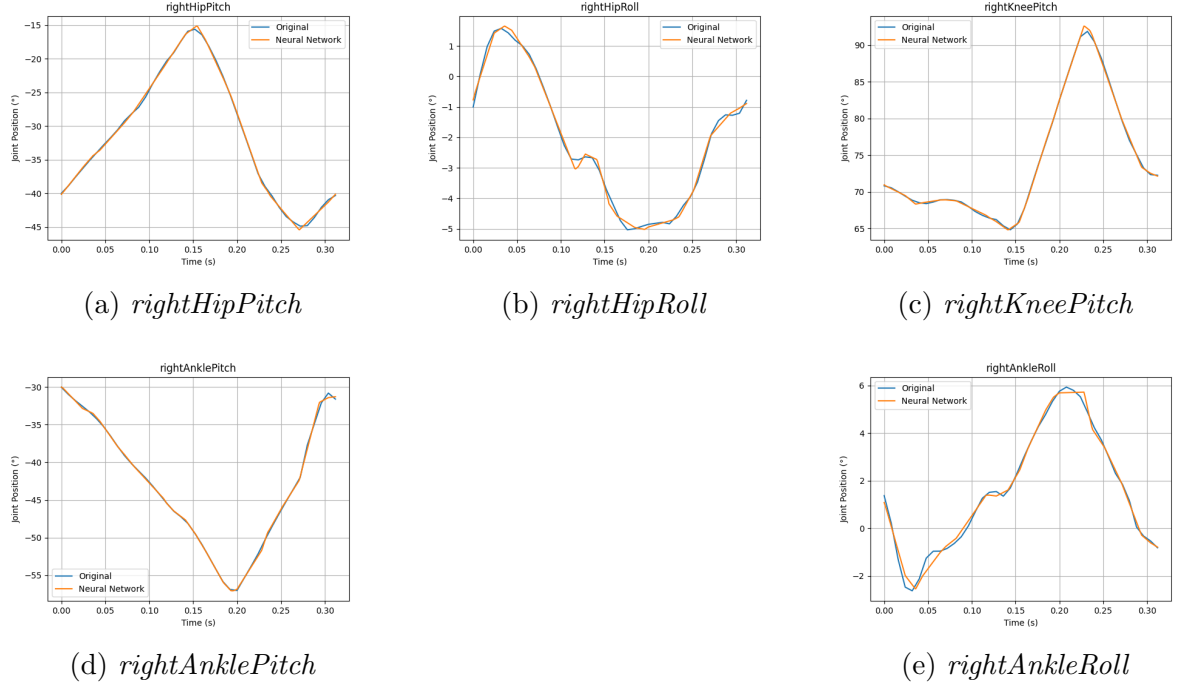


Figura 6: Trajetórias angulares previstas pela rede neural (em comparação com a referência) para as juntas do membro inferior direito ao longo do ciclo de marcha.

4 Conclusão

Este trabalho explorou a aplicação de redes neurais ao problema de reprodução do movimento de caminhada de um robô humanoide, por meio da técnica de *imitation learning*. Utilizando o *framework* PyTorch, foi implementada uma arquitetura *Multilayer Perceptron* capaz de mapear o tempo do ciclo de marcha às posições angulares das 20 juntas do robô, treinada com a função de custo MSE e o otimizador Adam.

Os resultados obtidos demonstram que a rede neural proposta é capaz de aproximar satisfatoriamente as trajetórias articulares fornecidas pelo controlador de referência, reproduzindo o padrão de marcha de forma consistente ao longo do ciclo. A análise das curvas para as juntas do membro inferior direito evidencia uma boa aderência entre a predição da rede e o sinal de referência, validando a abordagem adotada.

Adicionalmente, os experimentos com regularização nas funções `sum_gt_zero` e XOR permitiram ilustrar de forma prática o trade-off introduzido por esse mecanismo: embora a regularização produza fronteiras de decisão mais suaves e regulares, ela eleva o custo final de treinamento ao deslocar o ponto de mínimo da função objetivo. Em problemas determinísticos e sem ruído, como os investigados neste trabalho, esse efeito se manifesta como um viés indesejado, sem ganho correspondente em generalização — o que justifica a

escolha de não aplicar regularização ao treinamento da rede responsável pela marcha do robô.

De modo geral, o laboratório evidenciou a viabilidade do aprendizado por imitação como ferramenta para reprodução de movimentos em robôs humanoides, bem como a importância de adequar as escolhas de arquitetura e regularização à natureza específica do problema abordado.